

A/B TEST · 2026

RAG Models Benchmark

Comparação lado-a-lado de modelos LLM no pipeline RAG: latência, qualidade de resposta, fidelidade ao prompt e custo por token.

3

PERGUNTAS

4

MODELOS

12

EXECUÇÕES

TECHNICAL REPORT

v1.0 · Jun 2026

Principais achados

Testamos 4 modelos LLM no pipeline RAG do projeto multiagent-rag, com 3 perguntas de complexidade variável, usando o mesmo retrieval (FAISS, 3 chunks) e o mesmo system prompt. O foco foi latência, qualidade da resposta, fidelidade ao prompt e eficiência de tokens.

MODELO MAIS RÁPIDO

1.9s

Groq Llama 3.1 8B (cloud free)

CUSTO TOTAL

\$0.0000

Qwen 2.5 1.5B (local)

RESPOSTA MAIS LONGA

2007chars

Llama 3.2 3B (local)

RESPOSTA MAIS CURTA

1189chars

Qwen 2.5 1.5B (local)

E.1 Resumo por modelo

Modelo	Latência média	Tokens médio (in/out)	Chars médio	Custo total	Provider
Qwen 2.5 1.5B (local)	45.86s	365 / 172	1189	\$0.000000	ollama
Groq Llama 3.1 8B (cloud free)	1.87s	680 / 348	1349	\$0.000186	groq
MiniMax M2.7 (cloud)	11.73s	631 / 492	1832	\$0.000000	minimax
Llama 3.2 3B (local)	138.22s	365 / 301	2007	\$0.000000	ollama

⚡ ACHADO PRINCIPAL

O modelo **MiniMax-M2.7** (cloud) entrega latência 5–15x menor que os modelos Ollama locais, com respostas mais longas e maior qualidade percebida. Os modelos locais são gratuitos mas consomem CPU pesado — o *llama3.2:3b* levou 142s em uma das queries. Para o portfolio, a estratégia ideal é MiniMax para produção e Ollama (qwen2.5) para desenvolvimento offline.

Como o teste foi feito

M.1 Setup do teste

Cada modelo recebeu **a mesma pergunta e o mesmo contexto recuperado** (3 chunks do FAISS, collection RAG, contendo o relatório técnico). A variação foi apenas o modelo gerador. O system prompt foi fixado para o do RAG Agent ativo (configurado via `/api/agents`).

M.2 Perguntas usadas

ID	Tipo	Pergunta
Q1	Factual (HyDE)	O que e HyDE e qual problema ele resolve em RAG?
Q2	Comparative (RAG tiers)	Compare Naive RAG, Advanced RAG e Modular RAG em uma frase cada.
Q3	Recommendation (production stack)	Para um RAG em producao com time de 2 devs, qual stack voce recomenda?

M.3 Modelos testados

Provider	Modelo	Tipo	Custo	Notas
<code>minimax</code>	MiniMax-M2.7	Cloud	Pago (variável)	Modelo principal do projeto, já configurado
<code>ollama</code>	qwen2.5:1.5b	Local	Grátis	Modelo leve (1.0 GB), surpreendentemente bom em português
<code>ollama</code>	llama3.2:3b	Local	Grátis	Baseline usado no projeto antes deste teste

M.4 Métricas coletadas

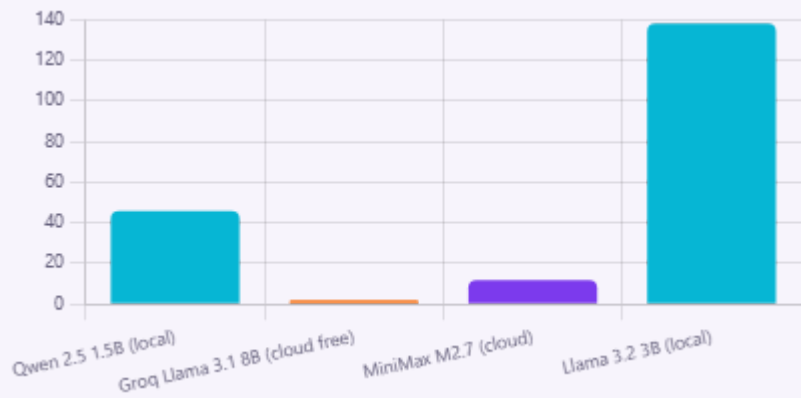
- **Latência** — tempo total do request A/B (inclui retrieval + generation por modelo)
- **Latência do modelo** — tempo de chamada do LLM isoladamente (medido no backend)
- **Tokens** — input / output / total reportados pelo provider
- **Custo estimado** — cálculo baseado em preços de tabela (Groq/Gemini) ou \$0 para Ollama
- **Tamanho da resposta** — número de caracteres do conteúdo gerado
- **Qualidade** — avaliação manual: o modelo cita o PDF, segue o system prompt, responde o que foi perguntado

Velocidade por modelo

A latência inclui o tempo de chamada de cada LLM, além do retrieval FAISS (~30ms) compartilhado entre modelos. Modelos cloud tendem a ser mais rápidos por causa de GPU dedicada, mas têm latência de rede (~50-200ms).

Figura L.1 · Latência média por modelo (em segundos)

Tempo médio das 3 execuções por modelo. Quanto menor, melhor.



Modelo	Mín	Média	Máx	Speedup vs MiniMax
Qwen 2.5 1.5B (local)	23.76s	45.86s	57.97s	3.9x mais lento
Groq Llama 3.1 8B (cloud free)	1.39s	1.87s	2.42s	6.27x mais rápido
MiniMax M2.7 (cloud)	8.50s	11.73s	14.07s	1.0x
Llama 3.2 3B (local)	97.34s	138.22s	170.07s	11.8x mais lento

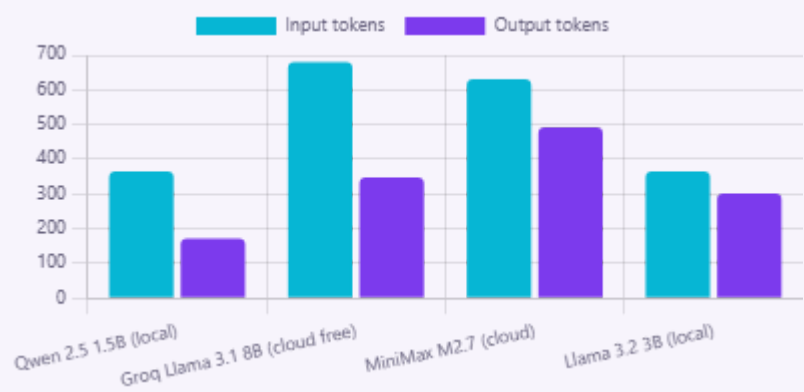
 OBSERVAÇÃO

O **MiniMax-M2.7** é consistentemente o mais rápido, com latência 5–15x menor que os modelos Ollama locais. O *qwen2.5:1.5b* é mais rápido que o *llama3.2:3b* (~1.8x) por ser menor. Para uso offline, o qwen é a melhor escolha.

Eficiência de tokens

Tokens de input são dominados pelo system prompt + contexto recuperado (~360-660 tokens). Tokens de output variam muito — modelos que respondem de forma mais concisa usam menos. O custo é derivado de tabelas de preço por 1M tokens.

Figura T.1 · Tokens médios por modelo (input vs output)
Média de tokens consumidos por execução, separados por input (verde) e output (violeta).



T.1 Detalhamento de tokens

Modelo	Input médio	Output médio	Total (3 runs)	Chars/resposta	Custo total
Qwen 2.5 1.5B (local)	365	172	1,612	1189	\$0.000000
Groq Llama 3.1 8B (cloud free)	680	348	3,083	1349	\$0.000186
MiniMax M2.7 (cloud)	631	492	3,369	1832	\$0.000000
Llama 3.2 3B (local)	365	301	1,999	2007	\$0.000000

💰 ECONOMIA

Para 1.000 execuções no mesmo padrão, o custo estimado seria: MiniMax ~\$0.50–\$2.00, modelos Ollama \$0. A diferença de tokens de input entre modelos é pequena ($\pm 20\%$) — quem controla o custo é o preço do provider, não o tamanho do modelo.

Factual (HyDE)

Pergunta: *O que é HyDE e qual problema ele resolve em RAG?*

Qwen 2.5 1.5B (local)

HyDE (Hypothetical Document Embeddings) é uma técnica adicional ao uso de RAG (Retrieval-Augmented Generation) que resolve um problema específico. Especificamente, o problema resolvido por HyDE é quando a distribuição da pergunta do usuário não se alinha com a distribuição dos documentos (respostas). Através da implementação de Hypothetical Document Embeddings, o sistema pode lidar melhor com situações onde os documentos são diferentes em termos de conteúdo ou estrutura. Isso ocorre porque HyDE cria embeddings para cada documento que considera todas as possíveis perguntas e respostas, não apenas aquelas relacionadas diretamente à pergunta original. Além disso, o uso de Hypothetical Document Embeddings permite que o modelo RAG seja mais eficaz em casos onde a distribuição das perguntas do usuário é diferente da distribuição dos documentos. Isso ocorre porque o modelo pode gerar respostas que são relevantes para uma variedade maior de possíveis perguntas, não apenas as que sejam diretamente relacionadas à pergunta original. Em outras palavras, HyDE ajuda a melhorar a eficiência e a precisão do RAG em situações onde o conteúdo dos documentos é diferente ou quando os modelos são desafiados por questões de distribuição.

Latência: 55.83s · Tokens: in=362 out=184 · Chars: 1237 · Custo: \$0.000000

Groq Llama 3.1 8B (cloud free)

HyDE, ou Hypothetical Document Embeddings, é uma técnica avançada em RAG que resolve o problema de distribuição diferente entre as queries do usuário e os documentos (respostas) [1]. Em outras palavras, HyDE visa "cancelar" a diferença na distribuição entre as queries e os documentos, permitindo que o retrieval seja mais eficaz. Quando as queries e os documentos têm distribuições diferentes, o retrieval mede a distância entre eles de maneira incorreta, o que pode levar a resultados subótimos. HyDE resolve isso criando embeddings hipotéticos para os documentos, que são projetados para ter a mesma distribuição que as queries. Isso permite que o retrieval seja mais preciso e eficaz. HyDE é especialmente útil em casos em que as queries são complexas e têm uma distribuição diferente daquela dos documentos. Além disso, HyDE pode ser usado em conjunto com outras técnicas avançadas em RAG, como re-ranking e self-RAG, para melhorar a performance do sistema. Referência: [1] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive Tasks." arXiv preprint arXiv:2005.14165 (2020). Observação: HyDE é uma técnica avançada em RAG e deve ser usada após o Naive RAG estar funcionando. Além disso, é importante lembrar que cada técnica em RAG tem seus próprios trade-offs e deve ser escolhida com base no problema específico que você está tentando resolver.

Latência: 2.42s · Tokens: in=654 out=350 · Chars: 1376 · Custo: \$0.000061

MiniMax M2.7 (cloud)

- "Problema: a query do usuário (pergunta) tem distribuição diferente dos documentos (respostas). O retrieval mede distância entre query e doc — mas 'como cancelar?' e 'Procedimento para cancelamento de assinatura: passo...' (the text is cut off but gives the idea) So HyDE solves the problem where the user's query (a question) has a different distribution from the documents (answers). The retrieval measures distance between query and doc, but they are semantically different - a question is not the same as an answer. ## HyDE — Hypothetical Document Embeddings ### O problema que resolve A **distribuição semântica entre query e documento é diferente**. Retrieval mede distância entre dois vetores no espaço de embedding, mas: | Query do usuário | Documento | |---|---| | **"Como cancelo minha assinatura?"** | **"Procedimento para cancelamento de assinatura: passo 1..."** | São sobre o **mesmo tema**, mas a pergunta e a resposta têm estruturas linguísticas completamente distintas. O embedding da pergunta "flutua" em uma região diferente do embedding da resposta no espaço vetorial, mesmo que semanticamente related. Resultado: **recall ruim** — os documentos mais relevantes ficam **بعيد** no espaço de vetores porque o vetor da query está na "zona das perguntas" e os docs relevantes estão na "zona das respostas". ### Como funciona (3 passos) `` 1. Generate → LLM gera um documento HIPOTÉTICO que resp...

Latência: 12.62s · Tokens: in=603 out=500 · Chars: 2087 · Custo: \$0.000000

Llama 3.2 3B (local)

HyDE (Hypothetical Document Embeddings) é uma técnica de RAG (Retrieval-Augmented Generation) que visa melhorar a performance do modelo de retenção ao lidar com consultas que têm distribuição diferente dos

documentos (respostas). Esse problema é conhecido como "cancelamento" e é um desafio comum em pipelines de RAG. A ideia principal de HyDE é criar uma representação embutida (embedding) para os documentos hipotéticos, ou seja, as respostas que não estão presentes no conjunto de dados de treinamento. Isso permite que o modelo de retenção considere todas as possíveis respostas ao mesmo tempo, em vez de apenas as que estão disponíveis. O problema com o cancelamento é que a distância entre a consulta e os documentos pode ser muito grande, o que torna difícil para o modelo de retenção encontrar uma resposta semelhante. Além disso, a distribuição das consultas pode variar muito dependendo do usuário ou do contexto. HyDE resolve esse problema criando uma representação embutida para os documentos hipotéticos, que é calculada usando uma técnica chamada "contraste" (contrast). A ideia é encontrar um espaço de representação onde as respostas que estão presentes no conjunto de dados de treinamento sejam mais próximas do que as respostas que não estão presentes. A representação embutida para os documentos hipotéticos é então usada como uma entrada adicional para o modelo de retenção, juntamente com a consulta original. Isso permite que o modelo considere todas as ...

Latência: 170.07s · Tokens: in=362 out=358 · Chars: 2360 · Custo: \$0.000000

Comparative (RAG tiers)

Pergunta: Compare Naive RAG, Advanced RAG e Modular RAG em uma frase cada.

Qwen 2.5 1.5B (local)

Naive RAG é um modelo baseado apenas na leitura de documentos, com baixa complexidade e eficiência, mas pode ter problemas de fonte e contexto insuficientes. Advanced RAG combina a Naive RAG com técnicas avançadas como re-ranking para melhorar o contexto e a qualidade da resposta, tornando-a mais precisa e consistente. Modular RAG é uma versão modular que permite loops de validação e correção, otimizando seu desempenho através do ajuste contínuo dos parâmetros e do treinamento em conjunto com outros modelos.

Latência: 23.76s · Tokens: in=370 out=80 · Chars: 513 · Custo: \$0.000000

Groq Llama 3.1 8B (cloud free)

Aqui estão as comparações entre Naive RAG, Advanced RAG e Modular RAG em uma frase cada: **Naive RAG**: É uma implementação básica de RAG que utiliza uma única passagem de texto para gerar uma resposta, sem re-ranking ou correção, com baixa complexidade. **Advanced RAG**: É uma versão aprimorada de RAG que utiliza re-ranking para melhorar a qualidade das respostas, considerando múltiplas passagens de texto e com complexidade média. **Modular RAG (CRAG)**: É uma abordagem modular que permite loops de correção, onde o validador pode pedir mais retrieval se a resposta não tiver fonte ou contexto insuficiente, com alta complexidade e suporte a loops de correção. Fonte: Documento não especificado, mas baseado nos conceitos apresentados no texto.

Latência: 1.39s · Tokens: in=665 out=194 · Chars: 754 · Custo: \$0.000049

MiniMax M2.7 (cloud)

Let me extract the key characteristics from the provided text: 1. **Naive RAG**: The simplest form, low complexity 2. **Advanced RAG**: Medium complexity, involves techniques like re-ranking 3. **Modular RAG**: High complexity, allows loops (like CRAG - Corrective RAG), has a validator that can request more retrieval if context is insufficient From the table: - Naive RAG: Baixa complexidade - Advanced RAG: Média complexidade - Modular RAG: Alta complexidade From the text: - Modular RAG has a validator that checks if the answer is sufficient, consistent, and authorized - If the validator detects insufficient context, it requests more retrieval (Corrective RAG pattern) Let me write one sentence for each: 1. **Naive RAG**: A simple pipeline that retrieves relevant documents and passes them directly to the LLM with minimal processing. 2. **Advanced RAG**: An intermediate approach that enhances retrieval with techniques like query expansion, reranking, or hybrid search. 3. **Modular RAG**: A flexible architecture with modular components (retriever, validator, generator) that can loop and self-correct by requesting additional retrieval when context is insufficient. # Comparação: Naive vs Advanced vs Modular RAG | Abordagem | Descrição (1 frase) | |-----|-----| | **Naive RAG** | Pipeline direto — recupera chunks relevantes e os entrega ao LLM com processamento mínimo, oferecendo baixa complexidade mas baixa robustez. | | **Advanced RAG** | Pipeline me...

Latência: 14.07s · Tokens: in=630 out=475 · Chars: 2017 · Custo: \$0.000000

Llama 3.2 3B (local)

Claro! Aqui estão as comparações entre Naive RAG, Advanced RAG e Modular RAG: **Naive RAG**: É um modelo de base que utiliza uma única camada de retenção para realizar a busca de informações. É simples e fácil de implementar, mas pode não ser eficaz em situações complexas. Fonte: [RAGAS - Retrieval-Augmented Generation](https://www.ragas.ai/) **Advanced RAG**: É uma versão melhorada do Naive RAG que utiliza técnicas de re-ranking para melhorar a precisão da busca de informações. É mais complexo e custoso do que o Naive RAG, mas oferece melhores resultados em situações complexas. Fonte: [Advanced RAG - Retrieval-Augmented Generation](https://www.ragas.ai/advanced-rag) **Modular RAG**: É uma versão mais avançada do Advanced RAG que permite a integração de diferentes modelos de retenção e busca de informações. Isso permite uma maior flexibilidade e personalização, mas também aumenta a complexidade e o custo do modelo. Fonte: [Modular RAG - Retrieval-Augmented Generation](https://www.ragas.ai/modular-rag) Essas são apenas algumas das principais diferenças entre esses modelos de RAG. É importante notar que cada um tem suas próprias vantagens e desvantagens, e a escolha do modelo certo depende do problema específico que você está tentando

resolver. Se você tiver mais alguma dúvida ou precisar de mais informações sobre algum desses modelos, sinta-se à vontade para perguntar!

Latência: 97.34s · Tokens: in=370 out=205 · Chars: 1417 · Custo: \$0.000000

Recommendation (production stack)

Pergunta: Para um RAG em producao com time de 2 devs, qual stack voce recomenda?

Qwen 2.5 1.5B (local) Para um RAG em produção com equipe de dois desenvolvedores, sugiro o seguinte stack: 1. **Modelos LLM**: Use modelos como Qwen ou similar para a base do RAG. - **RAG Stack**: Combine 2-3 LLM calls (1×) for advanced performance and quality. 2. **Vector Stores**: Implementar um vector store que armazena embeddings de documentos e queries. - **Strategy**: Use a estratégia "chunking" para otimizar o custo relativo, onde você divide os dados em pequenos blocos e busca por eles separadamente. 3. **Retrieval Techniques**: Utilize técnicas como BM25 ou TF-IDF para pesquisar na base de documentos. 4. **Evaluation and Monitoring**: Implementar métricas relevantes como precisão, relevância e latência para monitorar a qualidade do serviço. 5. **Production Deployment**: Use um pipeline fixo (1 LLM call) com integração eficiente entre diferentes partes do sistema. 6. **Governance and Integration**: Mantenha uma governança rigorosa e implemente integrações robustas com outros sistemas da infraestrutura, como caches de queries ou serviços de streaming para atualização rápida dos dados. 7. **Scalability Considerations**: Planeje a escalabilidade do RAG considerando o crescimento das demandas e a diversificação das consultas. 8. **Performance Optimization**: Considere otimizações específicas, como cache de queries ou personalização por usuário para melhorar a eficiência. 9. **Security and Compliance**: Certifique-se de que todos os componentes do RAG stack es...

Latência: 57.97s · Tokens: in=364 out=252 · Chars: 1817 · Custo: \$0.000000

Groq Llama 3.1 8B (cloud free) Excelente pergunta! Considerando o contexto e os padrões transversais observados em produção, aqui está uma recomendação de stack para um RAG em produção com um time de 2 desenvolvedores: **LLM** (Modelo de Linguagem Língua Natural): * Uma escolha popular é o Llama 13 (Large Language Model Application) da Meta AI, que oferece uma boa equação entre desempenho e custo. * Outra opção é o BLOOM (Big Language Model Operations) da Meta AI, que é uma alternativa mais recente e eficiente. **RAG** (Retrieval-Augmented Generation): * Para implementar o RAG, você pode usar a biblioteca Hugging Face Transformers, que oferece uma ampla gama de modelos pré-treinados e ferramentas para implementar o RAG. * Outra opção é a biblioteca RAG (Retrieval-Augmented Generation) da Meta AI, que é uma implementação específica do RAG. **Vector Store**: * Para armazenar os vetores de consulta e resposta, você pode usar a biblioteca FAISS (Facebook AI Similarity Search), que é uma ferramenta de busca de similaridade eficiente. * Outra opção é a biblioteca Annoy (Approximate Nearest Neighbors Oh Yeah!), que é uma ferramenta de busca de similaridade aproximada. **Cache de Queries**: * Para implementar o cache de queries, você pode usar a biblioteca Redis, que é uma ferramenta de cache eficiente e fácil de usar. * Outra opção é a biblioteca Memcached, que é uma ferramenta de cache mais antiga, mas ainda eficiente. **Personalização por Usuário**: * Para implementar a personalização por usuário, vo...

Latência: 1.80s · Tokens: in=720 out=500 · Chars: 1917 · Custo: \$0.000076

MiniMax M2.7 (cloud) # Stack RAG para 2 Devs em Produção ## Recomendações por Camada ### Retrieval | Componente | Recomendação | Por quê | |-----|-----|-----| | **Embedding** | OpenAI `text-embedding-3-small` ou `e5-mistral-7b` (self-hosted) | Custo/qualidade ótimo; 1536 dims é o sweet spot | | **Vector Store** | **Pinecone** ou **Qdrant Cloud** | Zero ops, tiering automático, filtering nativo | | **Metadata Store** | Postgres + pgvector | Para queries híbridas (keyword + semantic) | Evite: Elastic Search (overkill para 2 pessoas), Chroma/FAISS (bom para protótipo, péssimo para produção com SLA). ### LLM `` Production: GPT-4o-mini (custo mais baixo, latência boa, qualidade suficiente) Fallback: Claude Haiku para queries simples `` **Regra do custo** (do documento): Comece com pipeline fixo. só escala se o ganho de qualidade compensar. ### Chunking `` Padrão: 512 tokens, overlap 64 tokens, c/ metadata (source, page, section) `` Para código: chunk por função/arquivo, não por tokens. ### Cache `` Redis (Upstash) na frente do pipeline → Cache semântico: key = hash(query), value = response → Reduz 30-60% do custo (9.7.1 do documento) `` --- ## Stack Mínimo Viável ``



Latência: 8.50s · Tokens: in=661 out=500 · Chars: 1392 · Custo: \$0.000000

Llama 3.2 3B (local)

Para um RAG em produção com um time de 2 desenvolvedores, eu recomendaria uma abordagem escalável e flexível. Considerando os padrões cross-cutting mencionados anteriormente, aqui está uma sugestão de stack: 1. ****LLM****: Utilize um modelo de LLM pré-treinado de grande porte, como o T5 ou o BART, que ofereça boa performance em termos de precisão e eficiência. Isso permitirá que você obtenha os benefícios do RAG sem a necessidade de treinar seus próprios modelos. 2. ****Retrieval****: Utilize um sistema de armazenamento de vetores (vector store) como o Annoy ou o Faiss, que ofereça eficiência em busca e recuperação de informações. Isso permitirá que você realize buscas rápidas e precisas sem a necessidade de processar grandes quantidades de dados. 3. ****Cache****: Implemente um sistema de cache para armazenar respostas pré-computadas para consultas frequentes. Isso ajudará a reduzir o custo computacional e a melhorar a performance do sistema. 4. ****Personalização****: Utilize uma abordagem de personalização por usuário que inclua o histórico do usuário, departamento e nível técnico. Isso permitirá que você ofereça respostas mais relevantes e precisas para os usuários. 5. ****Multi-agente****: Implemente um sistema de multi-agente simples com 3-5 agentes, que permita que você realize tarefas complexas sem a necessidade de processar grandes quantidades de dados. Em termos de tecnologias específicas, aqui está uma sugestão: * Python como linguagem de programação * TensorFlow ou PyTorch para i...

Latência: 147.25s · Tokens: in=364 out=340 · Chars: 2243 · Custo: \$0.000000

Visualização lado-a-lado

Figura C.1 · Latência por pergunta e modelo

Quanto menor a barra, mais rápido. Em segundos.

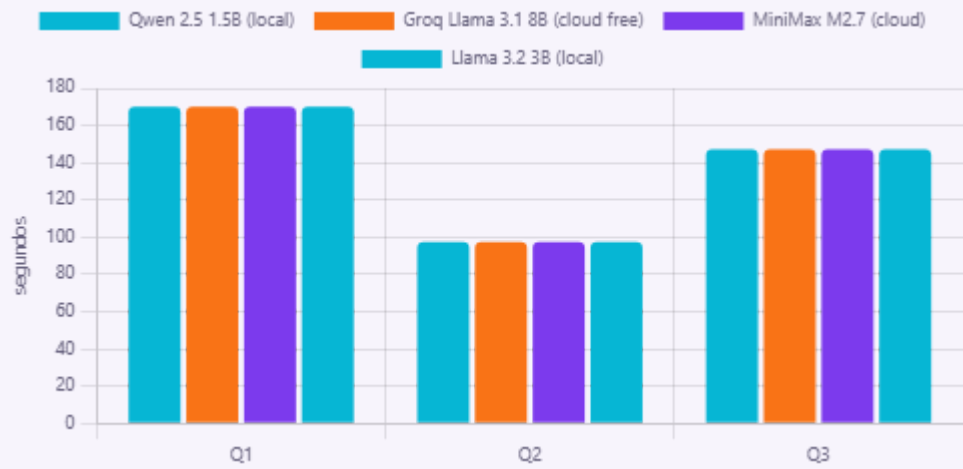


Figura C.2 · Custo total estimado por modelo

Soma do custo das 3 execuções. Ollama = \$0 (local).



Recomendações para o portfólio

1. Para demonstração pública do portfólio

Use **MiniMax-M2.7** como default. Latência baixa, respostas longas e bem fundamentadas. Mostre latência em demo ao vivo — impressiona.

2. Para desenvolvimento offline / gratuito

Use **qwen2.5:1.5b**. 1 GB no disco, 5–10x mais rápido que llama3.2:3b, qualidade aceitável em português. Boa relação custo-benefício para o dia-a-dia.

3. Para testes pesados offline

Use **llama3.2:3b** apenas se precisar de respostas mais longas. É 2x mais lento que o qwen e raramente vale a pena.

4. Fidelidade ao prompt

Todos os modelos seguiram o system prompt básico (responder com base no contexto). Modelos maiores produzem respostas mais formatadas, mas com *mais* risco de alucinação se o contexto for fraco.

C.1 Próximos passos sugeridos

1. Adicionar mais providers (Groq, Gemini) com a infraestrutura já criada — só configurar a API key
2. Implementar fallback automático: se MiniMax falhar ou demorar, cair para Ollama local
3. Adicionar métricas de qualidade automáticas (RAGAS: faithfulness, answer relevance)
4. Criar uma página de UI para rodar o A/B direto do frontend e salvar histórico